CAMPUS RECRUITMENT TRAINING & PLACEMENT BOARD

OFFICIAL PRESCRIBED PLACEMENT MASTER TEXTBOOK

Operating Systems & Concurrency Architecture: Placement Master Edition (Prescribed Textbook)

Subject Track: Operating Systems (OS)

Edition: 2026 Campus Placement Edition

SYLLABUS & CURRICULUM OVERVIEW

Authoritative campus placement textbook covering Operating Systems kernel architecture, process scheduling, synchronization primitives, deadlocks, virtual memory management, paging, and file systems.

STRICT EXAMINATION GROUNDING GUARANTEE

All campus placement assessments, live code challenges, milestone tests, and company interview drills for Operating Systems (OS) on this platform are derived strictly and exclusively from the theoretical concepts, algorithms, worked examples, and placement problems documented in this official textbook. Mastery of these chapters guarantees 100% exam preparation.

PRESCRIBED CHAPTER SYLLABUS (8 CHAPTERS):

Ch 1: OS Architecture, Kernel Dual Mode & System Calls

Ch 2: Process Management, Threads & Context Switching

Ch 3: CPU Scheduling Algorithms

Ch 4: Process Synchronization & Classical IPC Problems

Ch 5: Deadlock Handling & Banker's Algorithm

Ch 6: Memory Management, Paging & Segmentation

Ch 7: Virtual Memory & Page Replacement Algorithms

Ch 8: File Systems, Disk Scheduling & Solved Placement OS Questions

Author: Prescribed Courseware Architectural Board

Verified Complete Unabridged Placement Master Edition | Internal Campus Circulation

Edition: 2026 Campus Placement Edition

Subject: Operating Systems (OS)

Courseware Board: Prescribed Placement Architectural Board

Status: In-App Complete Master Textbook (Full Curriculum Available In-Platform)

> **? EXAM GROUNDING POLICY:** All campus placement technical assessments, coding lab challenges, and technical interview evaluations are strictly grounded in the concepts, algorithms, formulas, step-by-step solved examples, practice questions, and interview questions detailed in this textbook.

· Course Overview & Curriculum Scope

Authoritative campus placement textbook covering Operating Systems kernel architecture, process scheduling, synchronization primitives, deadlocks, virtual memory management, paging, and file systems.

· Complete Table of Contents (Chapter-by-Chapter)

- [Chapter 1: OS Architecture, Kernel Dual Mode & System Calls](#chapter-1)
- [Chapter 2: Process Management, Threads & Context Switching](#chapter-2)
- [Chapter 3: CPU Scheduling Algorithms](#chapter-3)
- [Chapter 4: Process Synchronization & Classical IPC Problems](#chapter-4)
- [Chapter 5: Deadlock Handling & Banker's Algorithm](#chapter-5)
- [Chapter 6: Memory Management, Paging & Segmentation](#chapter-6)
- [Chapter 7: Virtual Memory & Page Replacement Algorithms](#chapter-7)
- [Chapter 8: File Systems, Disk Scheduling & Solved Placement OS Questions](#chapter-8)

CHAPTER 1

OS Architecture, Kernel Dual Mode & System Calls

Key Principle / Exam Takeaway: Master Monolithic vs Microkernel architecture, User Mode vs Kernel Mode privilege rings, interrupt handling, and system calls.

Core Curriculum Topics Covered in this Chapter:

- Operating system roles: Hardware abstraction, resource manager, protection boundary
- Dual-Mode operation: User Mode (Ring 3) vs Kernel Mode (Ring 0)
- System Calls: Trap/Software Interrupt, Context switch from User to Kernel
- Monolithic Kernels (Linux) vs Microkernels (Mach, QNX)
- Hardware interrupts, software exceptions, and interrupt vector tables

· 1. Core Concepts & In-Depth Explanation

An Operating System acts as an intermediary between user applications and physical hardware.

Dual-Mode Operation: To prevent rogue applications from crashing the system or corrupting disk sectors, modern CPUs enforce hardware privilege levels (Dual-Mode operation):

- **User Mode (Ring 3):** Applications execute with restricted CPU instructions. Direct access to I/O devices, hardware registers, and raw memory is blocked.
- **Kernel Mode (Ring 0):** The OS kernel executes with full CPU privileges, accessing all hardware instructions and memory.

When an application requires hardware services (such as reading a file via `read()` or creating a process via `fork()`), it triggers a **System Call** (software interrupt/trap instruction). The CPU saves registers, transitions to Kernel Mode, executes the vetted kernel routine from the Interrupt Vector Table, and returns execution back to User Mode.

· 2. Key Formulas, Rules & Complexity Classes

1. **System Call Traversal:** $$\text{User Mode (App)} \xrightarrow{\text{trap / syscall}} \text{Kernel Mode (Syscall Handler)} \xrightarrow{\text{sysret}} \text{User Mode}$$

· 3. Step-by-Step Solved Examples

Example 1.1: Tracing System Call Execution Flow during file open()

- **Difficulty / Target Companies:** `Easy | Microsoft, TCS, Infosys`
- **Problem Statement:** Trace the step-by-step CPU transitions and register state changes when a C program invokes `open('file.txt', O_RDONLY)`.

Step-by-Step Walkthrough & Logic:

- Step 1: The standard library (glibc) places system call number (e.g., 2 on x86_64) into register `%rax` and arguments into `%rdi`, `%rsi`.

- Step 2: Glibc executes the `syscall` machine instruction.
- Step 3: The CPU hardware saves the user program counter (`%rip`) and flags, switches the stack pointer to the kernel stack, and sets CPU privilege level to Ring 0 (Kernel Mode).
- Step 4: The kernel's system call dispatcher indexes the `sys_call_table` and runs `sys_open()`.
- Step 5: The kernel places the returned File Descriptor into `%rax` and executes `sysret`, restoring User Mode.

Complete Solved Code / Calculation:

```
// Minimal C demonstration of direct syscall
#include <unistd.h>
#include <sys/syscall.h>

int main() {
    char msg[] = "Hello System Call!
";
    // Invoking write(1, msg, 19) directly via syscall
    syscall(SYS_write, 1, msg, 19);
    return 0;
}
```

- **Line-by-Line Breakdown & Complexity Analysis:** Demonstrates the direct hardware trap into the kernel without runtime overhead.

· 4. Practice Questions with Answers & Explanations

Practice Question 1.1

Question: What happens if a user-mode application executes a privileged CPU instruction such as CLI (Clear Interrupts)?

Options:

- A) The instruction is executed successfully
- B) CPU raises a General Protection Fault / Trap, terminating the process
- C) System freezes
- D) Process is demoted

Answer: **B) CPU raises a General Protection Fault / Trap, terminating the process** **Detailed Solution & Reasoning:** The hardware CPU checks the privilege level in the status register; executing privileged instructions in User Mode triggers a hardware exception.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 1.1 (Microsoft)

- **Interview Question:** _What is the difference between a Monolithic Kernel and a Microkernel?_

- **Model High-Scoring Answer:** In a Monolithic Kernel (e.g., Linux), all core OS services?process management, virtual memory, device drivers, and file systems?run in a single large address space in Kernel Mode. This provides blazing performance because internal communication occurs via fast direct C function calls, but a single buggy driver can crash the entire system. In a Microkernel (e.g., QNX), only minimal mechanisms (IPC, basic scheduling, virtual memory paging) run in Kernel Mode; servers (file systems, networking, drivers) run as isolated User Mode processes communicating via message passing. Microkernels are fault-tolerant and modular, but suffer higher IPC context-switch overhead.

- **Interviewer Follow-Up & Pro Tip:** ? Cite Linux as Monolithic and QNX/macOS Mach as Microkernel examples.

CHAPTER 2

Process Management, Threads & Context Switching

Key Principle / Exam Takeaway: Master Process Control Block (PCB), thread states, fork() vs exec(), context switch overhead, and User-level vs Kernel-level threads.

Core Curriculum Topics Covered in this Chapter:

- Process definition, memory segments (Text, Data, BSS, Heap, Stack)
- Process Control Block (PCB): PID, CPU registers, PC, scheduling state, open files
- Process creation and lifecycle: fork(), exec(), wait(), and exit()
- Zombie processes vs Orphan processes
- Thread model: Shared address space vs private stack, context switch costs

· 1. Core Concepts & In-Depth Explanation

A **Process** is a program in execution. Its address space consists of:

- **Text:** Compiled machine code instructions.
- **Data & BSS:** Initialized and uninitialized global/static variables.
- **Heap:** Dynamically allocated memory (`malloc`, `new`).
- **Stack:** Local variables, parameters, and return addresses.

The OS represents each process via a **Process Control Block (PCB)**.

Threads vs Processes: A Thread is the basic unit of CPU utilization ('lightweight process'). All threads within a single process share the identical address space, text, data, and open file descriptors, but each thread retains its own private Program Counter, register set, and Thread Stack.

Context Switch: Saving the state of an active process/thread (registers, stack pointer, page table base register CR3) and loading the state of a new process. Thread context switching is significantly faster than process switching because threads share the same virtual memory page tables, avoiding expensive TLB (Translation Lookaside Buffer) flushes.

· 2. Key Formulas, Rules & Complexity Classes

1. **Context Switch Performance Cost:** $$\text{Cost}(\text{Process Switch}) \gg \text{Cost}(\text{Thread Switch}) \quad \text{(Due to TLB invalidation)}$$

· 3. Step-by-Step Solved Examples

Example 2.1: Preventing Zombie Processes using waitpid()

- **Difficulty / Target Companies:** `Medium | Amazon, Microsoft, TCS`
- **Problem Statement:** Explain what a Zombie process is and write C code demonstrating how a parent reaps a child process.

Step-by-Step Walkthrough & Logic:

- Step 1: When a child process terminates, its memory is freed, but its exit status remains in the OS process table until read by parent.

- Step 2: If the parent never calls `wait()`, the child remains a 'Zombie'.
- Step 3: Call `waitpid()` in parent to reap exit status and clean PCB.

Complete Solved Code / Calculation:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();
    if (pid == 0) {
        // Child Process
        printf("Child exiting...
");
        _exit(0);
    } else {
        // Parent Process reaps child
        int status;
        waitpid(pid, &status, 0); // Reaps Zombie child
        printf("Parent reaped child successfully.
");
    }
    return 0;
}
```

- **Line-by-Line Breakdown & Complexity Analysis:** waitpid() reads the termination status from the kernel PCB table, removing the zombie entry.

· 4. Practice Questions with Answers & Explanations

Practice Question 2.1

Question: What is an Orphan Process and which process adopts it?

Options:

- A) A process with no memory, adopted by kernel
- B) A process whose parent terminated before it, adopted by the init process (PID 1)
- C) A dead process, adopted by garbage collector
- D) A process running in background

Answer: **B) A process whose parent terminated before it, adopted by the init process (PID 1)** **Detailed Solution & Reasoning:** When a parent dies before its child, the operating system re-parents the orphan to PID 1 (init/systemd), which periodically calls wait() to reap terminating orphans.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 2.1 (Google)

- **Interview Question:** _What is the difference between fork() and vfork() or clone() in Linux?_

- **Model High-Scoring Answer:** `fork()` historically duplicated the entire address space of the parent process, though modern Linux implements **Copy-On-Write (COW)**: parent and child share physical pages marked read-only until either modifies a page, whereupon the OS allocates a fresh copy. `clone()` is the underlying Linux system call that creates both processes and threads, accepting flags (`CLONE_VM`, `CLONE_FS`, `CLONE_FILES`) controlling what resources are shared.

- **Interviewer Follow-Up & Pro Tip:** ? Highlight Copy-On-Write (COW) as the primary optimization behind modern fork().

CHAPTER 3

CPU Scheduling Algorithms

Key Principle / Exam Takeaway: Master Turnaround vs Waiting vs Response time, Preemptive vs Non-Preemptive, Round Robin time slice tuning, and Multilevel Feedback Queues.

Core Curriculum Topics Covered in this Chapter:

- Scheduling criteria: CPU Utilization, Throughput, Turnaround Time, Waiting Time, Response Time
- First-Come, First-Served (FCFS) and the Convoy Effect
- Shortest Job First (SJF) and Shortest Remaining Time First (SRTF) optimal bounds
- Round Robin (RR): Time quantum selection and starvation immunity
- Priority Scheduling, Starvation, Aging techniques, and Multilevel Feedback Queues (MLFQ)

· 1. Core Concepts & In-Depth Explanation

CPU Scheduling selects an executable process from the Ready Queue to assign to the CPU core.

Key Metrics:

- **Turnaround Time (TAT):** Completion Time - Arrival Time.
- **Waiting Time (WT):** Turnaround Time - Burst Time.
- **Response Time (RT):** Time from arrival until first CPU execution.

Algorithms:

- **FCFS (Non-preemptive):** Suffers from the **Convoy Effect** (short processes queue behind long CPU-bound processes).
- **SJF / SRTF (Preemptive):** Mathematically proves minimal average waiting time. Suffers from starvation of long jobs.
- **Round Robin (Preemptive):** Each process gets a fixed time quantum q. If q is too large, it degenerates to FCFS; if q is too small, context-switch overhead dominates.
- **Multilevel Feedback Queue (MLFQ):** Dynamically adjusts process priorities: CPU-heavy jobs drop to lower queues with larger quanta, while I/O-interactive jobs stay in high-priority queues.

· 2. Key Formulas, Rules & Complexity Classes

1. **Scheduling Metrics:** $$TAT = \text{Completion Time} - \text{Arrival Time}$$ $$WT = TAT - \text{Burst Time}$$ 2. **Round Robin Time Quantum Balance:** $$\text{Rule of Thumb: } 80\% \text{ of CPU bursts should be shorter than time quantum } q$$

· 3. Step-by-Step Solved Examples

Example 3.1: Gantt Chart & Average Waiting Time Calculation (Round Robin)

- **Difficulty / Target Companies:** `Medium | Amazon, TCS, Infosys`

- **Problem Statement:** Processes P1 (Burst 24ms), P2 (Burst 3ms), P3 (Burst 3ms) arrive at time 0. Time quantum $q = 4$ms. Compute average waiting time.

Step-by-Step Walkthrough & Logic:

- Step 1: Gantt Chart execution: P1 (0-4), P2 (4-7, terminates), P3 (7-10, terminates), P1 (10-30, terminates).
- Step 2: Completion Times: P2 = 7, P3 = 10, P1 = 30.
- Step 3: Waiting Times: P1 = 30 - 24 = 6ms, P2 = 7 - 3 = 4ms, P3 = 10 - 3 = 7ms.
- Step 4: Average Waiting Time: $(6 + 4 + 7) / 3 = 17 / 3 = 5.67$ms.

Complete Solved Code / Calculation:

```
// Comparison of Average Waiting Times:
// FCFS (P1, P2, P3): (0 + 24 + 27) / 3 = 17.0 ms
// Round Robin (q = 4): (6 + 4 + 7) / 3 = 5.67 ms
// Significant reduction in waiting time for short interactive processes!
```

- **Line-by-Line Breakdown & Complexity Analysis:** Round Robin prevents the convoy effect by pre-empting P1 after 4ms, allowing P2 and P3 to complete quickly.

· 4. Practice Questions with Answers & Explanations

Practice Question 3.1

Question: What is the primary danger if the Round Robin time quantum is set to 1 microsecond?

Options:

- A) Starvation
- B) Context switch overhead consumes almost all CPU cycles, crippling useful throughput
- C) Deadlock
- D) Memory fragmentation

Answer: **B) Context switch overhead consumes almost all CPU cycles, crippling useful throughput** **Detailed Solution & Reasoning:** If the quantum approaches the context switch time, the CPU spends most of its time swapping registers rather than executing instructions.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 3.1 (Amazon)

- **Interview Question:** _How does Multilevel Feedback Queue (MLFQ) achieve high responsiveness without knowing burst times in advance?_
- **Model High-Scoring Answer:** MLFQ uses multiple priority queues with differing quanta (highest priority has smallest quantum). Rule 1: When a job enters the system, it is placed in the highest queue. Rule 2: If a job exhausts its time slice without yielding CPU, it is demoted to a lower queue (identifying it as CPU-bound). Rule 3: If a job yields CPU before quantum expires (e.g., waiting for keyboard I/O), it remains at high priority. Rule 4 (Aging): Periodically boost all jobs to top queue to prevent starvation.
- **Interviewer Follow-Up & Pro Tip:** ? Clearly articulate the 4 core MLFQ rules.

CHAPTER 4

Process Synchronization & Classical IPC Problems

Key Principle / Exam Takeaway: Master Critical Section, Race Conditions, Mutex vs Semaphore, Producer-Consumer, and Dining Philosophers.

Core Curriculum Topics Covered in this Chapter:

- Critical Section problem: Mutual Exclusion, Progress, Bounded Waiting
- Peterson's Algorithm for two-process synchronization
- Hardware synchronization primitives: Test-And-Set, Compare-And-Swap (CAS)
- Mutex Locks vs Counting Semaphores (wait/P and signal/V)
- Classical problems: Producer-Consumer (Bounded Buffer), Readers-Writers, Dining Philosophers

· 1. Core Concepts & In-Depth Explanation

When concurrent processes or threads access shared mutable data without coordination, a **Race Condition** occurs, where the final output depends on non-deterministic execution ordering.

A valid solution to the Critical Section problem must satisfy three criteria: 1. **Mutual Exclusion:** If process P_i is executing in its critical section, no other process can enter. 2. **Progress:** If no process is in the critical section, only processes wishing to enter participate in deciding who enters next. 3. **Bounded Waiting:** A bound must exist on the number of times other processes can enter before a requesting process is granted entry (prevents starvation).

Synchronization Tools:

- **Mutex (Mutual Exclusion):** Binary lock owned by a single thread at a time.
- **Counting Semaphore:** Integer counter tracking resource units. `wait()` (or `P()`) decrements; if counter < 0, process sleeps. `signal()` (or `V()`) increments and wakes a waiting process.

· 2. Key Formulas, Rules & Complexity Classes

1. **Semaphore Operation Semantics:** $$\text{wait}(S): \quad S.\text{count}--; \quad \text{if } (S.\text{count} < 0) \implies \text{block process in queue}$$ $$\text{signal}(S): \quad S.\text{count}++; \quad \text{if } (S.\text{count} \le 0) \implies \text{wake a blocked process}$$

· 3. Step-by-Step Solved Examples

Example 4.1: Producer-Consumer Solution with Semaphores (Bounded Buffer)

- **Difficulty / Target Companies:** `Medium | Amazon, Google, Microsoft`
- **Problem Statement:** Implement the Producer-Consumer pattern for a buffer of size N using semaphores.

Step-by-Step Walkthrough & Logic:

- Step 1: Semaphore `empty = N` (tracks available buffer slots).
- Step 2: Semaphore `full = 0` (tracks available items to consume).
- Step 3: Mutex `mutex = 1` (guarantees exclusive buffer access).

Complete Solved Code / Calculation:

```
// Producer:
wait(empty);  // Decrement empty slot count
wait(mutex);  // Lock buffer
// ... add item to buffer ...
signal(mutex);// Unlock buffer
signal(full); // Increment full slot count (wakes consumer)

// Consumer:
wait(full);   // Wait for at least one item
wait(mutex);  // Lock buffer
// ... remove item from buffer ...
signal(mutex);// Unlock buffer
signal(empty);// Increment empty slot count (wakes producer)
```

- **Line-by-Line Breakdown & Complexity Analysis:** Guarantees no buffer overflow, no underflow, and strict mutual exclusion during buffer modification.

· 4. Practice Questions with Answers & Explanations

Practice Question 4.1

Question: What is the key difference between a Mutex and a Binary Semaphore?

Options:

- A) Mutex is faster
- B) Mutex has ownership (only the thread that locked it can unlock it); a Semaphore can be signaled by any thread
- C) Semaphores only work on Linux
- D) Mutex allows 2 threads

Answer: **B) Mutex has ownership (only the thread that locked it can unlock it); a Semaphore can be signaled by any thread**
Detailed Solution & Reasoning: Mutex enforces strict ownership, which prevents priority inversion and unauthorized unlocking.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 4.1 (Google)

- **Interview Question:** _How do you prevent deadlock in the Dining Philosophers problem?_
- **Model High-Scoring Answer:** Deadlock occurs when all 5 philosophers pick up their left fork simultaneously and wait indefinitely for their right fork (Circular Wait). Solutions: 1) **Asymmetric Ordering:** Odd philosophers pick up left fork first, then right; even philosophers pick up right fork first, then left. This breaks the circular wait condition. 2) **Resource Hierarchy:** Impose a global ordering on forks (0 to 4); philosophers must always acquire the lower-numbered fork first. 3) **Arbitrator/Monitor:** Allow a philosopher to pick up forks only if both forks are simultaneously available.

- **Interviewer Follow-Up & Pro Tip:** ? Mention breaking the Coffman Circular Wait condition.

CHAPTER 5

Deadlock Handling & Banker's Algorithm

Key Principle / Exam Takeaway: Master the 4 Coffman conditions, Resource Allocation Graphs (RAG), Banker's Algorithm safety check, and deadlock detection.

Core Curriculum Topics Covered in this Chapter:

- Deadlock definition vs Livelock vs Starvation
- The 4 Coffman Conditions: Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait
- Resource Allocation Graph (RAG) and cycle detection criteria
- Deadlock Prevention vs Deadlock Avoidance
- Banker's Algorithm: Safety Algorithm and Resource-Request Algorithm

· 1. Core Concepts & In-Depth Explanation

A **Deadlock** is a state where a set of processes are blocked because each process holds resources and waits for other resources held by other processes in the set.

The 4 Coffman Conditions (Must ALL hold simultaneously for deadlock): 1. **Mutual Exclusion:** At least one resource is held in a non-shareable mode. 2. **Hold and Wait:** A process holds at least one resource and waits to acquire additional resources held by other processes. 3. **No Preemption:** Resources cannot be forcibly taken; they can only be released voluntarily by the holding process. 4. **Circular Wait:** A closed chain of processes exists $\{P_0, P_1, \dots, P_n\}$ where P_0 waits for P_1, and P_n waits for P_0.

Banker's Algorithm (Dijkstra): Used for **Deadlock Avoidance**. When a process requests resources, the OS simulates the allocation and tests if the resulting state is **Safe** (guaranteeing that at least one sequence of process executions exists where all processes can finish). If unsafe, the request is denied.

· 2. Key Formulas, Rules & Complexity Classes

1. **Banker's Need Matrix:** $$\text{Need}[i][j] = \text{Max}[i][j] - \text{Allocation}[i][j]$$ 2. **Safety Condition:** $$\text{Find } P_i \text{ such that } \text{Finish}[i] == \text{false and } \text{Need}[i] \le \text{Available}$$ $$\text{Work} = \text{Work} + \text{Allocation}[i]; \quad \text{Finish}[i] = \text{true}$$

· 3. Step-by-Step Solved Examples

Example 5.1: Executing Banker's Safety Algorithm

- **Difficulty / Target Companies:** `Medium | Amazon, TCS, Infosys`
- **Problem Statement:** Given 5 processes (P0-P4) and 3 resource types (A:10, B:5, C:7). Available = [3, 3, 2]. Determine if the system is in a safe state.

Step-by-Step Walkthrough & Logic:

- Step 1: Compute Need Matrix for each process: Need = Max - Allocation.
- Step 2: Compare Available [3, 3, 2] against Need: P1's Need [1, 2, 2] <= [3, 3, 2]. P1 can finish!

- Step 3: Reclaim P1's allocation [2, 0, 0]: New Available = [3, 3, 2] + [2, 0, 0] = [5, 3, 2].
- Step 4: Next, P3's Need [0, 1, 1] <= [5, 3, 2]. P3 finishes; Available = [5, 3, 2] + [2, 1, 1] = [7, 4, 3].
- Step 5: Continue for P4, P0, P2. All processes finish.
- Step 6: Safe sequence found: <P1, P3, P4, P0, P2>. System is SAFE.

Complete Solved Code / Calculation:

```
# Safe Sequence Verification:
# P1 -> P3 -> P4 -> P0 -> P2
# Since all processes can complete, the system state is strictly Safe!
```

- **Line-by-Line Breakdown & Complexity Analysis:** Because a valid safe execution sequence exists, deadlock is impossible under current allocation.

· 4. Practice Questions with Answers & Explanations

Practice Question 5.1

Question: If a Resource Allocation Graph contains a cycle and all resource types have exactly ONE instance, does a deadlock exist?

Options:

- A) No
- B) Yes, a cycle in a single-instance RAG is a necessary and sufficient condition for deadlock
- C) Only if processes are multithreaded
- D) Depends on priority

Answer: **B) Yes, a cycle in a single-instance RAG is a necessary and sufficient condition for deadlock** **Detailed Solution & Reasoning:** For single-instance resource types, a cycle directly implies circular wait and deadlock.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 5.1 (Amazon)

- **Interview Question:** _Why is the Banker's Algorithm rarely used in modern general-purpose operating systems like Linux or Windows?_
- **Model High-Scoring Answer:** Banker's Algorithm requires processes to declare their maximum resource requirements in advance, which is impossible for dynamic applications (like web browsers). Furthermore, the number of active processes and available resources is constantly fluctuating. In practice, modern OSes use the Ostrich Algorithm (ignoring deadlocks because they are rare) or rely on timeouts and process termination.
- **Interviewer Follow-Up & Pro Tip:** ? Mention the Ostrich Algorithm and inability to know maximum future resource claims.

CHAPTER 6

Memory Management, Paging & Segmentation

Key Principle / Exam Takeaway: Master Logical vs Physical addresses, MMU translation, Page Tables, TLB hits/misses, and multi-level paging.

Core Curriculum Topics Covered in this Chapter:

- Logical (Virtual) vs Physical address spaces, Memory Management Unit (MMU)
- Contiguous allocation: Internal vs External fragmentation, compaction
- Paging architecture: Pages, Frames, Page Table Entries (PTE)
- Translation Lookaside Buffer (TLB): Effective Memory Access Time (EMAT)
- Multi-level hierarchical paging and inverted page tables

· 1. Core Concepts & In-Depth Explanation

Memory Management isolates processes while maximizing RAM utilization.

Paging: Paging divides virtual memory into fixed-size blocks called **Pages** (typically 4KB) and physical memory into identical blocks called **Frames**. A hardware **Memory Management Unit (MMU)** uses a **Page Table** to translate virtual addresses to physical frame addresses:

- Virtual Address = `[ Page Number (p) | Offset (d) ]`
- Physical Address = `[ Frame Number (f) | Offset (d) ]`

Translation Lookaside Buffer (TLB): Because accessing the page table in RAM adds memory lookup latency, the CPU incorporates a fast associative hardware cache called the **TLB**. On memory access: 1. MMU checks TLB: If hit (99% of time), physical address is produced in < 1 cycle. 2. If TLB miss, MMU traverses Page Table in RAM, loads mapping into TLB, and completes access.

· 2. Key Formulas, Rules & Complexity Classes

1. **Virtual Address Decomposition (4KB Page Size = 2^{12} bytes):** $$\text{Offset } d = 12 \text{ bits}; \quad \text{Page Number } p = \text{Total Bits} - 12$$ 2. **Effective Memory Access Time (EMAT):** $$\text{EMAT} = h \times (t_{\text{TLB}} + t_{\text{RAM}}) + (1 - h) \times (t_{\text{TLB}} + 2 \times t_{\text{RAM}})$$ Where $h = \text{TLB Hit Ratio}$.

· 3. Step-by-Step Solved Examples

Example 6.1: Calculating Effective Memory Access Time (EMAT)

- **Difficulty / Target Companies:** `Easy | Amazon, TCS, Infosys`
- **Problem Statement:** Given TLB access time = 20ns, RAM access time = 100ns, and TLB hit ratio = 90%. Calculate EMAT.

Step-by-Step Walkthrough & Logic:

- Step 1: On TLB Hit: Time = $t_{TLB} + t_{RAM} = 20 + 100 = 120$ns.
- Step 2: On TLB Miss: Time = $t_{TLB} + 2 imes t_{RAM} = 20 + 200 = 220$ns (one RAM read for Page Table, one for actual data).

- Step 3: $ ext{EMAT} = 0.90 imes 120 + 0.10 imes 220 = 108 + 22 = 130$ns.

Complete Solved Code / Calculation:

```
// EMAT Formula:
// EMAT = 0.90 * (20 + 100) + 0.10 * (20 + 2 * 100)
// EMAT = 108 + 22 = 130 ns
```

- **Line-by-Line Breakdown & Complexity Analysis:** A 90% TLB hit ratio reduces average access time close to single RAM latency.

· 4. Practice Questions with Answers & Explanations

Practice Question 6.1

Question: What type of memory fragmentation occurs in a Paging system?

Options:

- A) External Fragmentation only
- B) Internal Fragmentation on the final allocated page
- C) Both Internal and External Fragmentation
- D) Zero fragmentation

Answer: **B) Internal Fragmentation on the final allocated page** **Detailed Solution & Reasoning:** Paging eliminates external fragmentation completely, but if a process needs 5KB with 4KB pages, it receives two 4KB pages (8KB total), leaving 3KB internal fragmentation.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 6.1 (Google)

- **Interview Question:** _Why do modern 64-bit systems use Multi-Level Paging (e.g., 4-level or 5-level paging) instead of a single flat page table?_
- **Model High-Scoring Answer:** A single flat page table for a 64-bit address space would require 2^{52} entries (exceeding petabytes of RAM per process) even if the process uses only 10MB of memory! Multi-Level Paging organizes page tables hierarchically into a tree. If a large region of virtual address space is unused, the entire intermediate branch in the page table tree is omitted (`NULL`), allowing processes to consume memory only for actively mapped pages.

- **Interviewer Follow-Up & Pro Tip:** ? Connect multi-level paging to sparse virtual address allocation.

CHAPTER 7

Virtual Memory & Page Replacement Algorithms

Key Principle / Exam Takeaway: Master Demand Paging, Page Fault handling, FIFO, LRU, Optimal (MIN), Belady's Anomaly, and Thrashing.

Core Curriculum Topics Covered in this Chapter:

- Virtual memory benefits: Oversubscription of physical RAM, lazy loading
- Page Fault lifecycle: Trap, disk I/O, frame allocation, page table update
- Page Replacement Algorithms: FIFO, Optimal (Belady's MIN), Least Recently Used (LRU)
- Belady's Anomaly proof on FIFO queues
- Thrashing: Working Set Model and page fault frequency control

· 1. Core Concepts & In-Depth Explanation

Virtual Memory decouples logical memory from physical RAM, allowing systems to execute processes larger than physical memory through **Demand Paging** (loading pages into RAM only when accessed).

Page Fault Handling: When a thread accesses a virtual page whose valid bit is 0: 1. MMU raises a **Page Fault Exception** (hardware trap). 2. OS kernel suspends process and searches for free physical frame. 3. If no frame is free, a **Page Replacement Algorithm** selects a 'victim' page to evict to disk swap space. 4. Kernel issues asynchronous disk read to fetch missing page into frame. 5. Page table updated (valid bit = 1); CPU restarts the faulting instruction.

Belady's Anomaly: The counter-intuitive phenomenon where allocating *more* physical page frames results in *more* page faults. This occurs in FIFO page replacement, but is mathematically impossible in stack-based algorithms like LRU and Optimal.

· 2. Key Formulas, Rules & Complexity Classes

1. **Belady's Anomaly Condition:** $$\text{FIFO Page Replacement on sequence } [1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5]:$$ $$\text{With 3 frames: } 9 \text{ faults} \quad \implies \quad \text{With 4 frames: } 10 \text{ faults!}$$

· 3. Step-by-Step Solved Examples

Example 7.1: Comparing FIFO vs LRU Page Faults

- **Difficulty / Target Companies:** `Easy | Amazon, TCS, Infosys`
- **Problem Statement:** Reference string: 7, 0, 1, 2, 0, 3, 0, 4. Capacity = 3 frames. Compute page faults for LRU.

Step-by-Step Walkthrough & Logic:

- Step 1: 7 -> [7] (Fault 1)
- Step 2: 0 -> [7, 0] (Fault 2)
- Step 3: 1 -> [7, 0, 1] (Fault 3)
- Step 4: 2 -> Evicts 7 (least recently used) -> [2, 0, 1] (Fault 4)

- Step 5: 0 -> Hit! [2, 0, 1] (Updates 0's recency)
- Step 6: 3 -> Evicts 1 -> [2, 0, 3] (Fault 5)
- Step 7: 0 -> Hit!
- Step 8: 4 -> Evicts 2 -> [4, 0, 3] (Fault 6)
- Total Page Faults = 6.

Complete Solved Code / Calculation:

```
// LRU Performance: 6 faults
// FIFO Performance: 7 faults
// LRU preserves frequently referenced pages (like 0) in memory!
```

- **Line-by-Line Breakdown & Complexity Analysis:** LRU leverages temporal locality, drastically outperforming simple FIFO.

· 4. Practice Questions with Answers & Explanations

Practice Question 7.1

Question: What is Thrashing in an operating system?

Options:

- A) Disk failure
- B) When the system spends more time swapping pages in/out of disk than executing useful user instructions
- C) CPU overheating
- D) Memory leak

Answer: **B) When the system spends more time swapping pages in/out of disk than executing useful user instructions**
Detailed Solution & Reasoning: Thrashing occurs when the sum of working sets of all active processes exceeds physical RAM capacity.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 7.1 (Microsoft)

- **Interview Question:** _Why is the Optimal (OPT/MIN) page replacement algorithm not implementable in practical operating systems?_
- **Model High-Scoring Answer:** The Optimal algorithm requires evicting the page that will not be used for the longest period of time in the *future*. Since general-purpose operating systems cannot predict future memory access sequences of arbitrary programs, OPT cannot be implemented. It serves as a theoretical mathematical benchmark against which practical algorithms like LRU and Clock are compared.
- **Interviewer Follow-Up & Pro Tip:** ? Mention the Clock (Second Chance) algorithm as the practical hardware approximation of LRU.

CHAPTER 8

File Systems, Disk Scheduling & Solved Placement OS Questions

Key Principle / Exam Takeaway: Master Unix inodes, file allocation methods, disk scheduling (SSTF, SCAN, C-SCAN), and top placement technical scenarios.

Core Curriculum Topics Covered in this Chapter:

- File System architecture: Inodes, superblocks, directory structures
- File allocation methods: Contiguous, Linked, and Indexed Allocation
- Disk Scheduling Algorithms: FCFS, Shortest Seek Time First (SSTF), SCAN (Elevator), C-SCAN
- Crash consistency, journaling file systems (ext4), and fsync()
- Comprehensive placement OS interview rubrics

· 1. Core Concepts & In-Depth Explanation

File systems translate logical file byte streams into persistent disk sectors:

- **Unix Inode:** A data structure containing file metadata (permissions, owner, size, timestamps) and direct/indirect block pointers. Direct pointers point directly to data blocks; indirect and double-indirect pointers enable files to scale into gigabytes/terabytes.
- **Journaling:** To prevent file system corruption during sudden power losses, modern file systems (ext4) log metadata changes to a designated journal before committing changes to main disk structures.

Disk Scheduling: Optimizes physical hard drive head travel (seek time):

- **SSTF:** Greedily serves request closest to current head position. Suffers from starvation of distant sectors.
- **SCAN (Elevator):** Moves in one direction servicing requests until reaching disk boundary, then reverses direction.
- **C-SCAN (Circular SCAN):** Moves in one direction servicing requests, then immediately jumps back to the beginning without servicing requests on the return journey, providing uniform wait times.

· 2. Key Formulas, Rules & Complexity Classes

1. **C-SCAN Total Head Movement:** $$\text{Head travels to disk boundary, jumps to 0, and continues to highest remaining request}$$

· 3. Step-by-Step Solved Examples

Example 8.1: Calculating Inode Maximum File Size Capacity

- **Difficulty / Target Companies:** `Medium | Amazon, Microsoft, TCS Digital`
- **Problem Statement:** An inode has 12 direct pointers, 1 single-indirect pointer, and 1 double-indirect pointer. Block size is 4KB. Disk block pointers are 4 bytes. Calculate the maximum supported file size.

Step-by-Step Walkthrough & Logic:

- Step 1: Direct blocks: $12 imes 4 ext{KB} = 48 ext{KB}$.

- Step 2: Pointers per block: $4 ext{KB} / 4 ext{bytes} = 1024$ pointers.
- Step 3: Single-indirect: $1024 imes 4 ext{KB} = 4 ext{MB}$.
- Step 4: Double-indirect: $1024 imes 1024 imes 4 ext{KB} = 1,048,576 imes 4 ext{KB} = 4 ext{GB}$.
- Step 5: Total capacity $ pprox 4 ext{GB} + 4 ext{MB} + 48 ext{KB} pprox 4.004 ext{GB}$.

Complete Solved Code / Calculation:

```
// Maximum Inode Capacity:
// Direct: 48 KB
// Single Indirect: 4 MB
// Double Indirect: 4 GB
// Total: ~4.004 GB
```

- **Line-by-Line Breakdown & Complexity Analysis:** Demonstrates how multi-level indirect indexing enables Unix inodes to support massive file sizes.

· 4. Practice Questions with Answers & Explanations

Practice Question 8.1

Question: Which disk scheduling algorithm provides the most uniform waiting time for requests across all sectors?

Options:

- A) FCFS
- B) SSTF
- C) SCAN
- D) C-SCAN

Answer: **D) C-SCAN** **Detailed Solution & Reasoning:** C-SCAN treats cylinders as a circular list, eliminating directional bias favoring middle tracks.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 8.1 (Amazon)

- **Interview Question:** _What is the difference between a Hard Link and a Soft (Symbolic) Link in Unix file systems?_
- **Model High-Scoring Answer:** A Hard Link is an additional directory entry pointing directly to an existing file's **inode number**. Both directory entries share identical file metadata and data blocks; deleting one does not delete the file until the inode link count reaches zero. Hard links cannot span across different file systems. A Soft (Symbolic) Link is a distinct, separate file with its own unique inode containing the string pathname of the target file. If the target file is deleted, the symlink breaks ('dangling link'). Symlinks can cross file system boundaries.
- **Interviewer Follow-Up & Pro Tip:** ? Cite inode link count and cross-filesystem limitations.

· Campus Placement Certification & Final Exam Policy

To earn Placement-Ready Master status, students must complete all chapter practice questions, review all solved examples, and score at least 80% on the comprehensive subject assessment.